



**THE
UNIVERSITY OF
LAHORE**

Group Members:

70153904 Muhammad Saad Khan

70152951 Muhammad Shahrose Rafique

70152488 Junaid Ahmad

70149352 Mohsin Qureshi

Subject:

Database Management System

Degree:

BS-IET (5-I)

DBMS Project Proposal: TechFix Management System

1. Domain and Topic Selection

- **Selected Domain:** Retail & Technical Services
- **Project Topic:** Hardware Repair Shop Management System (TechFix)

2. Novelty of the Project

While traditional repair shops rely on paper receipts and manual ledgers, this project introduces a unified digital ecosystem. Its novelty lies in bridging inventory management with customer ticketing. It features automated real-time repair status tracking for customers and integrated threshold-based alerts for hardware spare parts, preventing bottlenecks caused by out-of-stock components.

3. Detailed Project Report

Introduction

The management of a hardware repair shop involves juggling customer intake, device diagnostics, technician assignments, spare parts inventory, and billing. This project aims to design and implement a robust relational database (using MySQL) to streamline these operations. The system ensures data integrity, speeds up retrieval times for customer inquiries, and optimizes inventory flow.

Problem Statement

Local hardware repair shops frequently face operational inefficiencies:

1. **Lost Device Context:** Paper tickets get misplaced, leading to confusion over what was wrong with a device.
2. **Inventory Mismanagement:** Technicians frequently run out of critical spare parts (like SSDs, screens, or specific ICs) because there is no system linking repair usage to stock depletion.
3. **Poor Customer Communication:** Customers must call or visit the shop to check their repair status, taking up valuable staff time.
4. **Inefficient Billing:** Calculating labor plus specific part costs manually leads to accounting errors.

Entities, Attributes, and Relationships

Entities and Attributes:

- **Customer:** CustomerID (PK), FirstName, LastName, Phone, Email
- **Device:** DeviceID (PK), CustomerID (FK), DeviceType (e.g., Laptop, Mobile), Brand, Model, SerialNumber
- **Technician:** TechID (PK), Name, Specialty, Phone
- **RepairTicket:** TicketID (PK), DeviceID (FK), TechID (FK), IssueDescription, Status, ReceiveDate, EstimatedCompletion, TotalLaborCost

- **Inventory:** PartID (PK), PartName, Category, QuantityInStock, UnitPrice
- **TicketParts (Bridge Table):** UsageID (PK), TicketID (FK), PartID (FK), QuantityUsed

Relationships:

- A **Customer** can own *many* **Devices** (1:N).
- A **Device** can have *many* **RepairTickets** over time (1:N).
- A **Technician** handles *many* **RepairTickets** (1:N).
- A **RepairTicket** can use *many* **Parts**, and a **Part** can be used in *many* **RepairTickets** (M:N, resolved by the TicketParts bridge table).

ERD (Entity-Relationship Diagram) Description

To map this out in your presentation or drawing tool (like draw.io or Lucidchart):

1. Place **Customer** on the left, connecting with a one-to-many line to **Device**.
2. Connect **Device** to **RepairTicket** with a one-to-many line.
3. Place **Technician** above **RepairTicket**, connecting downwards with a one-to-many line.
4. Place **Inventory** on the right. Connect it to **RepairTicket** using the bridging entity **TicketParts** in the middle (one-to-many from both RepairTicket and Inventory to TicketParts).

Database Constraints

To maintain data integrity, the following constraints are applied across the database:

- **Primary Keys (PK):** Unique identifiers for every entity (e.g., TicketID, PartID) ensure no duplicate records.
- **Foreign Keys (FK):** Maintain referential integrity. A RepairTicket cannot exist for a DeviceID that doesn't exist.
- **NOT NULL:** Critical fields like Customer.Phone, Device.Model, and RepairTicket.IssueDescription cannot be left empty.
- **CHECK Constraint:** The Status column in RepairTicket is restricted to specific values: CHECK (Status IN ('Pending', 'Diagnosing', 'Awaiting Parts', 'Repaired', 'Delivered')).
- **DEFAULT Values:** ReceiveDate defaults to the CURRENT_DATE. QuantityInStock defaults to 0.

System Flowchart / Use-Case Flow

1. **Intake Phase:** Customer brings in a broken laptop. Admin creates a Customer record (if new) and a Device record.
2. **Ticketing Phase:** Admin generates a RepairTicket, logging the issue and assigning it to an available Technician. Status is set to 'Pending'.

3. **Repair Phase:** Technician evaluates the device. Updates status to '*Diagnosing*'.
4. **Parts Requisition:** If parts are needed, the Technician logs them in TicketParts. The system checks Inventory. If stock is sufficient, QuantityInStock is deducted.
5. **Completion Phase:** Technician finishes the work, updates labor cost, and changes status to '*Repaired*'.
6. **Checkout:** Admin queries the total cost (Labor + Parts), bills the customer, and updates status to '*Delivered*'.

Output Screens / Query Results

Here are the core SQL queries that power the system's output screens.

1. Dashboard Screen: View all currently pending or active repairs

SQL

```
SELECT t.TicketID, c.FirstName, d.Brand, d.Model, t.IssueDescription, t.Status
FROM RepairTicket t
JOIN Device d ON t.DeviceID = d.DeviceID
JOIN Customer c ON d.CustomerID = c.CustomerID
WHERE t.Status NOT IN ('Repaired', 'Delivered')
ORDER BY t.ReceiveDate ASC;
```

Result Example:

TicketID	FirstName	Brand	Model	IssueDescription	Status
101	Ahmed	Dell	Latitude 5400	Screen flickering	Diagnosing
102	Sara	Infinix	Note 50	Charging port broken	Awaiting Parts

2. Inventory Screen: Low Stock Alert (Parts needing reorder)

SQL

```
SELECT PartID, PartName, Category, QuantityInStock
FROM Inventory
WHERE QuantityInStock < 5;
```

Result Example:

PartID	PartName	Category	QuantityInStock
503	256GB SATA SSD	Storage	2
512	Thermal Paste	Consumables	1

3. Checkout Screen: Generate Final Invoice for a specific Ticket (e.g., Ticket 101)

SQL

```
SELECT
    t.TicketID,
    t.TotalLaborCost,
    COALESCE(SUM(i.UnitPrice * tp.QuantityUsed), 0) AS TotalPartsCost,
```

```

    (t.TotalLaborCost + COALESCE(SUM(i.UnitPrice * tp.QuantityUsed), 0)) AS GrandTotal
FROM RepairTicket t
LEFT JOIN TicketParts tp ON t.TicketID = tp.TicketID
LEFT JOIN Inventory i ON tp.PartID = i.PartID
WHERE t.TicketID = 101
GROUP BY t.TicketID, t.TotalLaborCost;

```

Result Example:

TicketID	TotalLaborCost	TotalPartsCost	GrandTotal
101	2500.00	4500.00	7000.00

Conclusion

The TechFix Database Management System successfully eliminates the chaos of manual repair tracking. By organizing data into normalized relational tables, the shop can seamlessly track a device from the moment it is dropped off to the moment the invoice is paid. The integration of inventory tracking directly into the repair workflow ensures that technicians always have the hardware they need, ultimately leading to faster turnaround times and higher customer satisfaction.